

Los tipos de API que vas a encontrar

Todas mueven datos entre sistemas. La diferencia está en **cómo** hablan: el transporte, el formato, quién pide y quién empuja, y qué tan rígido es el contrato. Acá tenés cada estilo con un ejemplo real de request y response, y cuándo conviene o no.

● REST

HTTP · JSON ·
recursos

● SOAP

XML · contrato
WSDL

● GraphQL

1 endpoint ·
query

● gRPC

HTTP/2 · protobuf

● WebSockets

canal
bidireccional

● Webhooks

el server te
avisa

01 REST Representational State Transfer

El estándar de facto. Todo es un recurso con una URL, y usás los verbos de HTTP para operar sobre él.

REST no es un protocolo, es un estilo de diseño sobre HTTP. La idea central: cada cosa (un usuario, un pedido, una factura) tiene una dirección, y el método HTTP dice qué querés hacer. GET para leer, POST para crear, PUT/PATCH para modificar, DELETE para borrar. El servidor no guarda estado de tu sesión entre llamadas, así que cada request lleva todo lo que necesita.

REQUEST · LEER UN USUARIO

```
# pedís el recurso por su URL
GET /api/users/42 HTTP/1.1
Host: api.tienda.com
Authorization: Bearer eyJhbGci...
Accept: application/json
```

RESPONSE · 200 OK

```
{
  "id": 42,
  "nombre": "Carlos Rodríguez",
  "email": "carlos@ejemplo.com",
  "pedidos": "/api/users/42/pedidos" # enlace al recurso relacionado
}
```

REQUEST · CREAR UN PEDIDO

```
POST /api/users/42/pedidos HTTP/1.1
Content-Type: application/json

{ "producto_id": 991, "cantidad": 2 }

# respuesta: 201 Created + el pedido nuevo en el body
```

Lo que lo hizo ganar: es predecible. Si conocés una API REST, conocés casi todas. Se cachea gratis con la infraestructura web que ya existe (CDN,

proxies), se prueba con curl o Postman, y cualquier lenguaje sabe hablar HTTP y JSON.

USALO CUANDO

- Exponés una API pública o para terceros
- El modelo de datos entra bien en "recursos" (CRUD clásico)
- Querés aprovechar caché HTTP y herramientas estándar

EVITALO CUANDO

- El cliente pide muchos recursos anidados y termina haciendo 8 llamadas (over/under-fetching)
- Necesitás tiempo real o streaming continuo
- El contrato entre cliente y server tiene que ser estricto y tipado

02 SOAP Simple Object Access Protocol

El veterano rígido. Un protocolo formal, todo en XML, con un contrato que define cada operación antes de escribir una línea.

SOAP es anterior a REST y sigue vivo donde importa la formalidad: bancos, seguros, sistemas gubernamentales, ERPs viejos. Cada mensaje va envuelto en un "envelope" XML. El contrato lo define un archivo WSDL que describe operaciones, tipos y errores; a partir de él muchos lenguajes te generan el cliente automáticamente. Trae estándares para seguridad a nivel mensaje (WS-Security), transacciones y garantías de entrega.

REQUEST · POST A UN ENDPOINT ÚNICO

```
POST /servicio/cotizaciones HTTP/1.1
Content-Type: text/xml; charset=utf-8

<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <GetCotizacion>
      <moneda>USD</moneda>
    </GetCotizacion>
  </soap:Body>
</soap:Envelope>
```

RESPONSE

```
<soap:Envelope>
  <soap:Body>
    <GetCotizacionResponse>
      <valor>1350.50</valor>
    </GetCotizacionResponse>
  </soap:Body>
</soap:Envelope>
```

El costo es la verbosidad y la rigidez. El XML pesa, el envelope agrega ruido, y cambiar el contrato es un trámite. Pero esa misma rigidez es la razón por la que un sistema de pagos de hace 15 años sigue funcionando sin sorpresas.

USALO CUANDO

- Integrás con banca, seguros o sistemas legacy que ya hablan SOAP
- Necesitás seguridad y transacciones formales a nivel mensaje
- El contrato tiene que ser un documento firme y auditable

EVITALO CUANDO

- Arrancás algo nuevo sin obligación de compatibilidad
- Trabajás con apps móviles o web donde el peso importa
- Querés iterar rápido sin regenerar contratos

03 GraphQL creado por Facebook, 2015

Vos pedís exactamente los campos que querés, ni uno más. Un solo endpoint, y la respuesta tiene la forma de tu pregunta.

GraphQL nació para resolver el dolor de REST en apps con pantallas complejas: en vez de pegarle a `/users`, después a `/users/42/pedidos` y después a `/pedidos/7/items`, mandás una sola query que describe todo el árbol que necesitás. El server responde con esa estructura y nada más. Un solo endpoint (`/graphql`), casi siempre por POST, y un schema fuertemente tipado que documenta la API sola.

REQUEST · UNA QUERY PIDE LO JUSTO

```
POST /graphql
{
  usuario(id: 42) {
    nombre
    pedidos(ultimos: 2) {
      fecha
      items { producto cantidad }
    }
  }
}
```

RESPONSE · MISMA FORMA QUE LA QUERY

```
{ "data": {
  "usuario": {
    "nombre": "Carlos Rodríguez",
    "pedidos": [
      { "fecha": "2026-08-01", "items": [{ "producto": "Teclado", "cantidad": 1 } ]
    }
  }
}}
```

Ganás flexibilidad brutal para el frontend: cambiás la pantalla sin tocar el backend. El precio lo paga el servidor. El caché HTTP ya no aplica igual (todo es POST al mismo endpoint), una query mal armada puede pedir medio universo, y armar el backend con sus "resolvers" tiene más curva.

USALO CUANDO

- Tenés muchos clientes distintos (web, móvil) con necesidades de datos diferentes
- Las pantallas combinan datos de varias fuentes
- Querés que el frontend evolucione sin pedir endpoints nuevos

EVITALO CUANDO

- La API es simple y CRUD basta
- Dependés mucho del caché HTTP a nivel CDN
- No querés cargar con la complejidad del server y el control de queries

04 gRPC gRPC Remote Procedure Call · Google

No pensás en recursos ni URLs: llamás a una función remota como si fuera local. Rápido, binario, tipado, ideal entre servicios.

gRPC es un estilo RPC: definís métodos ("dame el usuario 42") en un archivo `.proto`, y una herramienta genera el código de cliente y server en el lenguaje que uses. Va sobre HTTP/2 y serializa con Protocol Buffers, un formato binario que pesa mucho menos que JSON. Soporta streaming en las dos direcciones. El costo: un navegador no lo consume directo (necesita un proxy), así que casi siempre vive puertas adentro, entre microservicios.

CONTRATO · EL ARCHIVO `.PROTO`

```
// define el servicio y los tipos, una sola fuente de verdad
service Usuarios {
  rpc GetUsuario (UsuarioReq) returns (Usuario);
}
message UsuarioReq { int32 id = 1; }
message Usuario { int32 id = 1; string nombre = 2; }
```

CLIENTE · LLAMÁS EL MÉTODO COMO UNA FUNCIÓN

```
# el código de abajo lo generó gRPC desde el .proto
resp = client.GetUsuario(id=42)
print(resp.nombre)    # "Carlos Rodríguez"
```

Entre servicios es difícil de superar: baja latencia, poco ancho de banda, contrato estricto que rompe en tiempo de compilación si alguien cambia algo. Por eso es el default para comunicación interna en arquitecturas de microservicios.

USALO CUANDO

- Comunicás microservicios internos y la performance cuenta
- Necesitás streaming (por ejemplo, telemetría o chat entre servicios)
- Querés contratos tipados que se validan al compilar

EVITALO CUANDO

- El cliente es un navegador y no querés montar un proxy
- Necesitás que terceros integren fácil con curl
- La legibilidad del payload en texto plano te importa para debug rápido

05 WebSockets canal persistente y bidireccional

HTTP es pedí-y-esperá. WebSockets abre un caño abierto en las dos direcciones: el server te empuja datos sin que preguntes.

En HTTP normal el cliente pregunta y el server responde, se corta, y para saber si hay algo nuevo tenés que volver a preguntar (polling).

WebSockets rompe eso: arranca con un "upgrade" sobre HTTP y a partir de ahí queda una conexión abierta por donde ambos lados mandan mensajes cuando quieren. Un chat, un ticker de precios, la posición de un delivery en un mapa, un documento colaborativo: todo eso vive acá.

HANDSHAKE · EL CLIENTE PIDE UPGRADE

```
GET /chat HTTP/1.1
Host: app.ejemplo.com
Upgrade: websocket
Connection: Upgrade
```

```
# server responde: 101 Switching Protocols
# de acá en más la conexión queda abierta
```

MENSAJES · EN CUALQUIER DIRECCIÓN, CUANDO PASA ALGO

```
// cliente -> { "tipo": "mensaje", "texto": "hola" }
// server -> { "tipo": "mensaje", "de": "Ana", "texto": "buenas" }
// server -> { "tipo": "escribiendo", "de": "Ana" } // nadie lo pidió
```

El costo es que la conexión abierta consume recursos y complica el escalado: cada usuario conectado es estado vivo en tu server, y hay que manejar reconexiones cuando se cae la red. Si solo necesitás que el server empuje (sin que el cliente responda por el mismo canal), Server-Sent Events es una alternativa más liviana.

USALO CUANDO

- Necesitás tiempo real de verdad (chat, juegos, colaboración en vivo)
- El server tiene que empujar cambios apenas ocurren
- El flujo es de ida y vuelta constante

EVITALO CUANDO

- Alcanza con pedir datos cada tanto (usá REST)
- Solo el server empuja y el cliente nunca responde (usá SSE)
- No querés lidiar con estado de conexiones y reconexiones

06 Webhooks "HTTP reverso" · callbacks

Al revés que todo lo demás: vos exponés una URL y el servicio ajeno te hace un POST cuando pasa algo que te interesa. Dejá de preguntar.

Un webhook no es un protocolo nuevo, es un patrón sobre HTTP. En vez de que tu app le pregunte a Stripe cada 5 segundos "¿ya pagó?", registrás una URL tuya y Stripe le pega un POST a esa URL en el momento exacto en que el pago se confirma. El evento llega solo. Es la forma normal de reaccionar a cosas que pasan en sistemas de terceros: pagos, mails abiertos, un push a GitHub, un mensaje entrante.

TU ENDPOINT RECIBE EL EVENTO

```
# Stripe le pega a la URL que registraste
POST /webhooks/stripe HTTP/1.1
Stripe-Signature: t=1725,v1=5a2f... # firma: verificá que es real

{
  "type": "payment_intent.succeeded",
  "data": { "amount": 2500, "currency": "usd" }
}

# respondé 200 rápido, y procesá aparte
```

Dos reglas que se aprenden a los golpes: **verificá la firma** (cualquiera podría postear a tu URL) y **respondé rápido con 200**, encolando el trabajo pesado, porque si tardás el proveedor reintenta y te llega el evento repetido. Por eso conviene que tu handler sea idempotente: el mismo evento dos veces no debe cobrar dos veces.

USALO CUANDO

- Tenés que reaccionar a eventos de un servicio externo (pagos, CI, mensajería)
- El polling sería un desperdicio o llegaría tarde
- Querés integraciones desacopladas y en tiempo casi real

EVITALO CUANDO

- Necesitás una respuesta sincrónica ya (pedí, no esperes un callback)
- No podés exponer una URL pública que reciba tráfico
- El orden y la entrega exacta de cada evento son críticos sin lógica extra

07 La tabla comparativa

Todo lo anterior, lado a lado.

ESTILO	TRANSPORTE	FORMATO	MODELO	CONTRATO	TIEMPO R
● REST	HTTP/1.1	JSON	Recursos + verbos	Informal (OpenAPI)	No
● SOAP	HTTP, otros	XML	Operaciones	Estricto (WSDL)	No
● GraphQL	HTTP (1 endpoint)	JSON	Query de campos	Estricto (schema)	Parcial (subscrip
● gRPC	HTTP/2	Protobuf (binario)	Llamada a función	Estricto (.proto)	Sí (stream
● WebSockets	TCP (upgrade HTTP)	Libre (JSON, binario)	Mensajes bidireccionales	Vos lo definís	Sí
● Webhooks	HTTP (POST entrante)	JSON	Evento empujado	Del proveedor	Casi real

Nota: no son mutuamente excluyentes. Un sistema real mezcla varios. REST para la API pública, gRPC entre los servicios internos, WebSockets para el chat y webhooks para escuchar a Stripe. Conviven todo el tiempo.

08 **Cómo elegir, rápido**

Cinco preguntas que resuelven el 90% de los casos.

¿API pública o para terceros que integren fácil?

REST. Es lo que todos esperan y lo que mejor documentan las herramientas.

¿El frontend necesita datos muy distintos según la pantalla?

GraphQL. Cada cliente pide lo suyo sin multiplicar endpoints.

¿Comunicación interna entre servicios, con foco en performance?

gRPC. Binario, tipado y con streaming. El estándar puertas adentro.

¿Tiempo real con ida y vuelta (chat, en vivo)?

WebSockets. Si solo empuja el server, mirá Server-Sent Events antes.

¿Reaccionar a algo que pasa en un servicio ajeno?

Webhooks. Registrás tu URL y el evento te llega solo. Verificá la firma.

¿Integrás con banca o un sistema viejo que ya lo pide?

SOAP. Rara vez lo elegís de cero, pero cuando toca, toca.

Guía de referencia · nivel práctico intermedio. Los ejemplos son ilustrativos y están simplificados para mostrar la forma de cada estilo, no para copiar y pegar en producción.